

Análise Global de Mudanças Climáticas com Apache Spark

Rodrigo Macedo e Gabriel Peres

2026

Resumo

Este relatório descreve um pipeline distribuído em Apache Spark para analisar dados climáticos e emissões de CO2. O projeto usa bases reais de temperatura global por cidade e emissões da OWID, executa limpeza, agregações, cruzamentos, funções de janela, cache, regressão com MLlib e geração de gráficos. A execução foi validada em Docker local e em dois computadores conectados por rede local, com acompanhamento pela Spark UI.

Sumário

1	Objetivo	3
2	Arquitetura e fluxo de comunicação	3
2.1	Portas usadas	4
2.2	Decisões de arquitetura	4
3	Bases usadas	4
4	Fluxo do pipeline	5
4.1	Limpeza de temperatura	5
4.2	Limpeza de CO2	5
4.3	Cruzamento dos dados	6
5	Respostas das questões	6
5.1	Questão 1: média anual global por década	6
5.2	Questão 2: anos mais quentes por continente	7
5.3	Questão 3: cidades em risco	8

5.4	Questão 4: correlação entre mínima e máxima em zonas tropicais	8
5.5	Questão 5: qualidade dos dados	8
5.6	Questão 6: aumento de CO2 versus aumento de temperatura	9
5.7	Questão 7: ranking de aceleração térmica	10
5.8	Questão 8: previsão com MLlib	10
6	Spark, paralelismo e conclusão prática	11
6.1	Onde o Spark divide o trabalho	11
6.2	Cache e reaproveitamento de etapas	12
6.3	Benchmark local versus cluster LAN	13
6.4	Workers, recursos e escalabilidade	14
6.5	Por que mais workers podem não reduzir o tempo	14
6.6	Conclusão a partir das medições	15

1 Objetivo

Este relatório apresenta o projeto desenvolvido para o trabalho proposto pelo professor. O foco da entrega é aplicar Apache Spark em um problema real de análise de dados, mostrando leitura de bases climáticas, limpeza, transformação, cruzamento entre temperatura e CO2, execução distribuída, respostas das questões pedidas e avaliação prática de desempenho.

Este PDF é o principal material de avaliação. Por isso, ele reúne o processo completo em formato explicativo: quais dados foram usados, por que cada tratamento foi feito, onde o Spark divide o processamento, como cada pergunta foi respondida e como validamos a execução em Docker local e em dois computadores. O notebook `notebooks/apresentacao_spark_clima.ipynb` fica como apoio executável para demonstrar o mesmo fluxo rodando passo a passo.

2 Arquitetura e fluxo de comunicação

O projeto usa PySpark com DataFrames. A aplicação é empacotada em Docker para que o mesmo código rode no modo local e no modo distribuído em dois computadores. Isso reduz diferenças de ambiente e facilita repetir a execução durante a apresentação.

No modo local, o arquivo `docker-compose.yml` sobe um master Spark e dois workers no mesmo computador. Esse modo é útil para testar o pipeline inteiro sem depender de rede externa. No modo distribuído, o PC 1 roda o master, um worker local e o driver do job. O PC 2 roda outro worker apontando para o master do PC 1:

```
spark://<IP_D0_PC_1>:7077
```

O fluxo de comunicação fica assim:

1. O driver inicia o job e conversa com o master Spark.
2. O master registra os workers disponíveis.
3. O driver monta o plano lógico e físico das operações.
4. O Spark quebra o plano em jobs, stages e tasks.
5. O master agenda tasks nos executores dos workers.
6. Os executores leem suas partições de dados, processam e devolvem resultados intermediários.
7. O driver coleta apenas os resultados pequenos que precisam virar tabela, gráfico ou arquivo final.

Tabela 1: Componentes da arquitetura

Componente	Papel no trabalho
Driver	Processo que submete o job, cria a <code>SparkSession</code> , chama as funções do pipeline e coordena as ações.
Master Spark	Serviço que conhece os workers vivos e agenda recursos para os executores.
Worker	Processo que oferece CPU e memória para o cluster. No teste em 2 PCs, há um worker em cada máquina.
Executor	Processo que roda tasks Spark dentro de um worker. É nele que as partições são processadas.
Spark UI	Interface usada para provar visualmente workers, executores, jobs, stages, tasks e duração das etapas.

2.1 Portas usadas

Tabela 2: Portas usadas no modo distribuído

Porta	Função
7077	Master Spark, usado pelos workers para conexão.
8080	Spark UI do master, usada para mostrar os workers vivos.
40444	Porta do driver do job.
40445	Block manager, usado em comunicação de blocos e dados intermediários.

2.2 Decisões de arquitetura

A primeira decisão foi manter os dados replicados nos dois computadores no modo raw. Cada executor lê arquivos locais montados em `/app/data/...`. Isso evita depender de HDFS, S3 ou outro storage distribuído, que aumentaria o escopo do trabalho. A consequência é que PC 1 e PC 2 precisam ter o mesmo diretório `data/raw` antes do benchmark.

A segunda decisão foi usar DataFrames em vez de RDDs manuais. DataFrames permitem que o Catalyst otimize filtros, agregações, joins e projeções. Para este trabalho, isso é mais adequado porque as perguntas são consultas analíticas sobre tabelas.

A terceira decisão foi salvar saídas pequenas em `output/results` e gráficos em `output/plots`. O processamento pesado fica no Spark. A conversão para pandas só acontece depois que os dados já foram agregados e estão pequenos o bastante para caber no driver.

3 Bases usadas

Foram usadas duas fontes principais:

- Temperatura: Berkeley Earth, arquivo `GlobalLandTemperaturesByCity.csv`.

- CO2: OWID, arquivo `owid-co2-data.csv`.

O arquivo `GlobalTemperatures.csv` também fica disponível no projeto, mas a resposta da questão sobre zonas tropicais usa a base por cidade, porque ela contém cidade, país, latitude e longitude. Essa escolha permite filtrar regiões tropicais diretamente pela latitude.

4 Fluxo do pipeline

O pipeline segue a ordem abaixo:

1. Ler os CSVs com Spark.
2. Limpar a base de temperatura.
3. Limpar a base de CO2.
4. Criar colunas auxiliares como ano, mês, década e país normalizado.
5. Agregar temperatura mensal por cidade para temperatura anual por país.
6. Cruzar temperatura e CO2 por país e ano.
7. Responder as perguntas com agregações, joins, window functions e MLlib.
8. Gerar tabelas, gráficos e arquivos de saída.

4.1 Limpeza de temperatura

A limpeza remove registros sem `AverageTemperature`, converte `dt` para `data`, extrai `year`, `month` e `decade`, converte coordenadas como `23.55S` para valores numéricos negativos e remove temperaturas fora de -90°C a 70°C . Esse limite é uma regra de sanidade: ele não pretende modelar clima, apenas evita que erro de leitura ou valor impossível distorça médias e desvios.

Também é criada a coluna `country_norm`. Ela normaliza o nome do país para permitir o join com CO2. Sem essa coluna, diferenças pequenas de caixa, espaço ou acento poderiam impedir cruzamentos corretos.

4.2 Limpeza de CO2

A base de CO2 passa por filtros para remover agregados como `World`, `Asia` e `Europe`. Esses registros não representam países e poderiam duplicar a escala da análise. Também são mantidos apenas anos a partir de 1900 e linhas com `co2` preenchido.

A mesma regra de `country_norm` é aplicada. Isso garante que o join use a mesma chave nas duas bases.

4.3 Cruzamento dos dados

Antes do join, a temperatura mensal por cidade é agregada para temperatura anual por país:

```
country_norm + year -> avg_temperature
```

Depois disso, o join com CO2 usa:

```
country_norm + year
```

Essa transformação muda a granularidade dos dados. A temperatura começa mensal e por cidade. O CO2 está anual e por país. Para cruzar as bases, as duas precisam chegar ao mesmo nível: país e ano.

5 Respostas das questões

5.1 Questão 1: média anual global por década

Primeiro calculamos a média anual global. Depois agrupamos os anos por década e aplicamos uma média móvel. A década reduz ruído anual. A média móvel ajuda a enxergar a tendência sem depender de um único ano muito frio ou muito quente.

Tabela 3: Média anual global por década

Década	Média anual	Média móvel
1980	17.9578	17.8420
1990	18.2497	18.0012
2000	18.5268	18.2448
2010	18.6335	18.4700

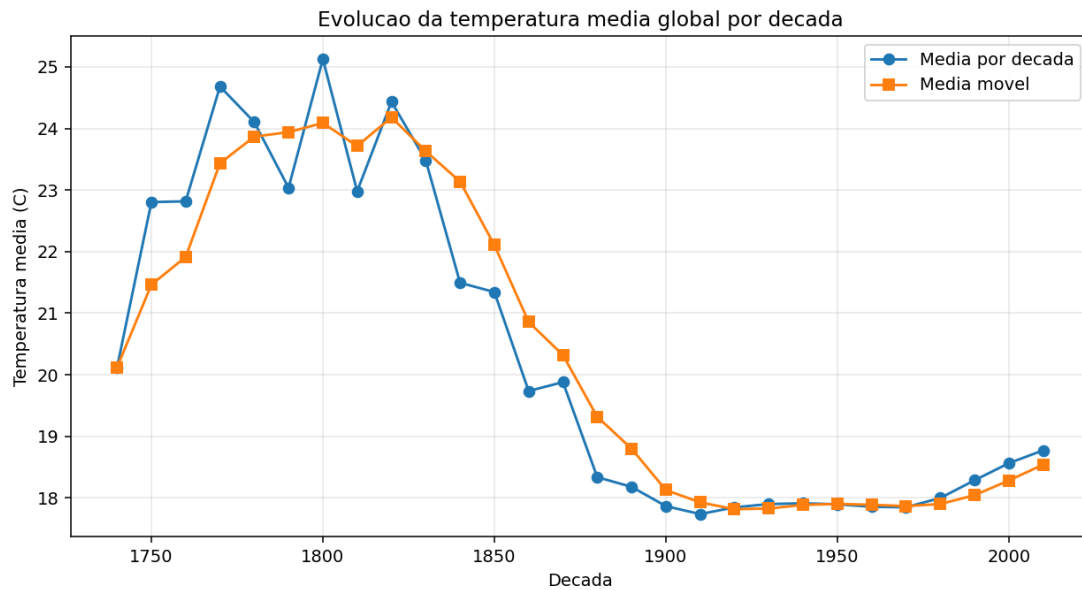


Figura 1: Evolução da temperatura média global por década.

O resultado mostra aumento nas décadas recentes. A leitura correta é de tendência agregada, não de previsão climática completa.

5.2 Questão 2: anos mais quentes por continente

A base original não vem com continente. Por isso, o pipeline usa um mapeamento de país para continente. Depois agrega temperatura por continente e ano, e usa `row_number()` para escolher o ano mais quente de cada continente.

A função de ranking é importante porque evita escolher apenas o maior valor global. Cada continente precisa ter seu próprio primeiro lugar.

Tabela 4: Ano mais quente por continente

Continente	Ano mais quente	Média
Africa	2010	24.1887
Asia	2013	21.3166
Europe	2007	9.9243
North America	2013	18.2733
Oceania	1998	16.3499
South America	2002	21.9883

A decisão arbitrária aqui é depender do mapeamento disponível de país para continente. Se um país não estiver mapeado, ele fica fora desse recorte. Isso muda a cobertura, mas mantém a pergunta tecnicamente correta para os países classificados.

5.3 Questão 3: cidades em risco

O critério do enunciado é o desvio padrão de temperatura. Por isso, a pergunta não mede maior aquecimento médio, e sim maior variação histórica de temperatura. As cidades no topo aparecem em regiões continentais frias, onde a amplitude sazonal costuma ser alta.

Tabela 5: Cidades com maior desvio padrão

Cidade	País	Desvio padrão
Heihe	Russia	16.4134
Blagoveshchensk	Russia	16.4134
Kyzyl	Russia	16.3454
Hailar	China	16.3026
Yakeshi	China	16.3026

Esse resultado deve ser explicado com cuidado: alto desvio padrão indica grande oscilação, não necessariamente a maior alta recente de temperatura. Se o critério mudasse para crescimento por década, o ranking poderia mudar.

5.4 Questão 4: correlação entre mínima e máxima em zonas tropicais

A base por cidade não possui colunas reais de mínima e máxima. Para responder mantendo o foco em zonas tropicais, filtramos cidades entre -23.5 e 23.5 de latitude. Depois, para cada cidade e ano, usamos uma aproximação:

- mínima anual aproximada: menor média mensal do ano;
- máxima anual aproximada: maior média mensal do ano.

Essa é a principal limitação da questão. O cálculo não usa mínima e máxima meteorológicas reais, e sim extremos entre médias mensais. A vantagem é que a resposta fica compatível com a base disponível. A diferença prática é que extremos diários são suavizados, então a correlação tende a representar comportamento anual agregado.

O Pearson encontrado foi:

```
0.4220268963949332
```

Isso indica correlação positiva moderada entre os meses mais frios e mais quentes nas cidades tropicais.

5.5 Questão 5: qualidade dos dados

A pergunta usa a incerteza da temperatura. A regra aplicada foi:

```
AverageTemperatureUncertainty > 0.10 * abs(media_historica_da_cidade)
```


O percentual de 10% é um corte definido para separar registros mais instáveis. Se esse percentual for reduzido, mais registros entram como alta incerteza. Se for aumentado, menos registros são marcados. Por isso, o notebook deixa essa variável fácil de alterar durante a apresentação.

Tabela 6: Registros com alta incerteza

Alta incerteza	Registros
true	1.608.419
false	6.626.663

A marcação de incerteza não remove os dados automaticamente. Ela cria uma coluna de qualidade para análise e discussão.

5.6 Questão 6: aumento de CO2 versus aumento de temperatura

Para cada país nos últimos 50 anos disponíveis, capturamos o primeiro e o último ano do período. Depois calculamos:

```
co2_delta = co2_final - co2_inicial
temp_delta = temp_final - temp_inicial
```

O Pearson entre os deltas foi:

```
0.07968357901250334
```

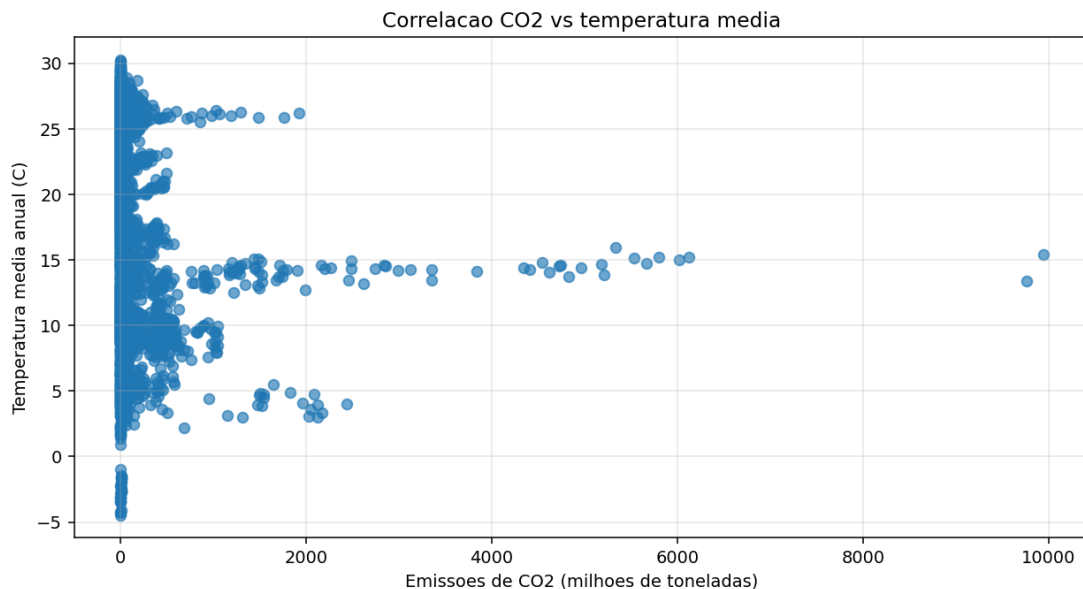


Figura 2: Dispersão entre variação de CO2 e variação de temperatura.

A correlação ficou fraca. Isso não invalida o processamento. A pergunta cruza duas bases reais e calcula uma relação estatística simples. A limitação é de interpretação: emissões absolutas por

país misturam população, economia, matriz energética, latitude, território e geografia. Isso mede associação no recorte escolhido, não causalidade climática.

5.7 Questão 7: ranking de aceleração térmica

A questão usa window functions com lag por país. Primeiro calculamos médias por década. Depois comparamos a última década completa com a anterior. Como a base Berkeley termina por volta de 2013, a última década completa usada foi 2000-2009.

Tabela 7: Países com maior aceleração térmica

País	Década	Aceleração
Azerbaijan	2000	0.4929
Kazakhstan	2000	0.4900
Uzbekistan	2000	0.4680
Tajikistan	2000	0.4657
Afghanistan	2000	0.4562

A decisão de usar apenas décadas completas evita comparar uma década cheia com uma década parcial. Se usássemos 2010-2013, a comparação poderia ficar artificial porque o período teria menos anos.

5.8 Questão 8: previsão com MLlib

Foi usado LinearRegression do Spark MLlib. A feature é o ano e o label é a temperatura média. Para Rio De Janeiro, Brazil, a base termina antes dos anos atuais. Por isso, os cinco anos previstos são os cinco anos seguintes ao último ano disponível.

Tabela 8: Previsão para Rio de Janeiro

Ano	Temperatura prevista
2014	20.5780
2015	20.5762
2016	20.5745
2017	20.5727
2018	20.5710

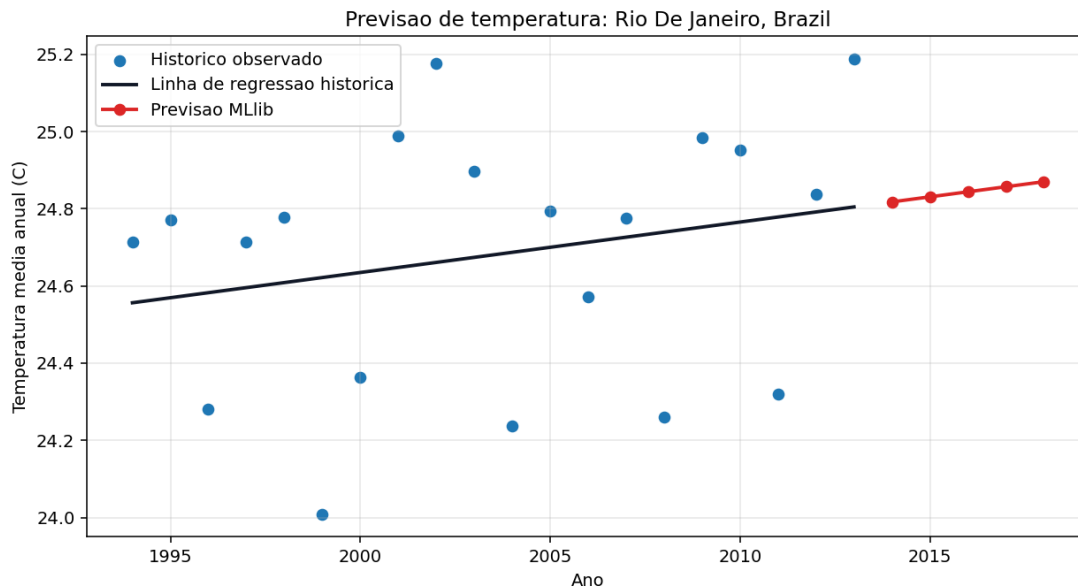


Figura 3: Previsão para Rio de Janeiro. Os pontos históricos não são conectados; a linha mostra a regressão e a projeção.

A regressão linear foi escolhida por ser simples de explicar e por existir no MLlib. Ela não pretende ser o melhor modelo climático possível. Se fossem usados mais atributos, como CO2, latitude, sazonalidade ou janelas temporais, a modelagem mudaria. Aqui o objetivo é mostrar o uso de MLlib no pipeline distribuído.

6 Spark, paralelismo e conclusão prática

Esta seção junta a leitura técnica do Spark com as medições feitas no projeto. Ela é a parte mais importante para defender que o trabalho distribui processamento, mas também para explicar por que distribuir não significa automaticamente reduzir tempo.

6.1 Onde o Spark divide o trabalho

A divisão acontece nas partições dos DataFrames. Quando o Spark lê arquivos CSV, ele cria partições. Operações como `groupBy`, `join`, `orderBy` e `window functions` geram stages e tasks. Cada task pode ser enviada para um executor disponível.

Na Spark UI, essa divisão aparece em:

- **Workers:** máquinas conectadas ao master.
- **Executors:** processos que executam tasks.
- **Jobs:** ações disparadas pelo código, como `collect`, `count` e escrita de arquivos.
- **Stages:** partes do plano separadas por shuffle.

- **Tasks:** menor unidade de execução distribuída.

Ver dois workers vivos na Spark UI confirma que o cluster está conectado. Para confirmar que o job usou os dois, também é preciso olhar executors, hosts, tasks e tempo de execução durante a rodada.

6.2 Cache e reaproveitamento de etapas

O modo `-cache on` persiste DataFrames reutilizados:

- temperatura limpa;
- CO2 limpo;
- temperatura anual por país;
- temperatura anual por cidade;
- join entre temperatura e CO2.

O cache é útil porque várias perguntas usam as mesmas bases intermediárias. Sem cache, o Spark pode recomputar leitura, limpeza e agregações várias vezes.

Tabela 9: Tempo por pergunta com e sem cache

Pergunta	Com cache (s)	Sem cache (s)	Ganho aproximado
Q1	2.3362	5.5426	2.37x
Q2	3.9009	12.3879	3.18x
Q3	1.7852	12.7051	7.12x
Q4	2.4746	12.0828	4.88x
Q5	0.9610	14.3473	14.93x
Q6	0.6194	14.3876	23.23x
Q7	4.3224	24.9529	5.77x
Q8	3.9533	22.0286	5.57x
Total	20.3530	118.4348	5.82x

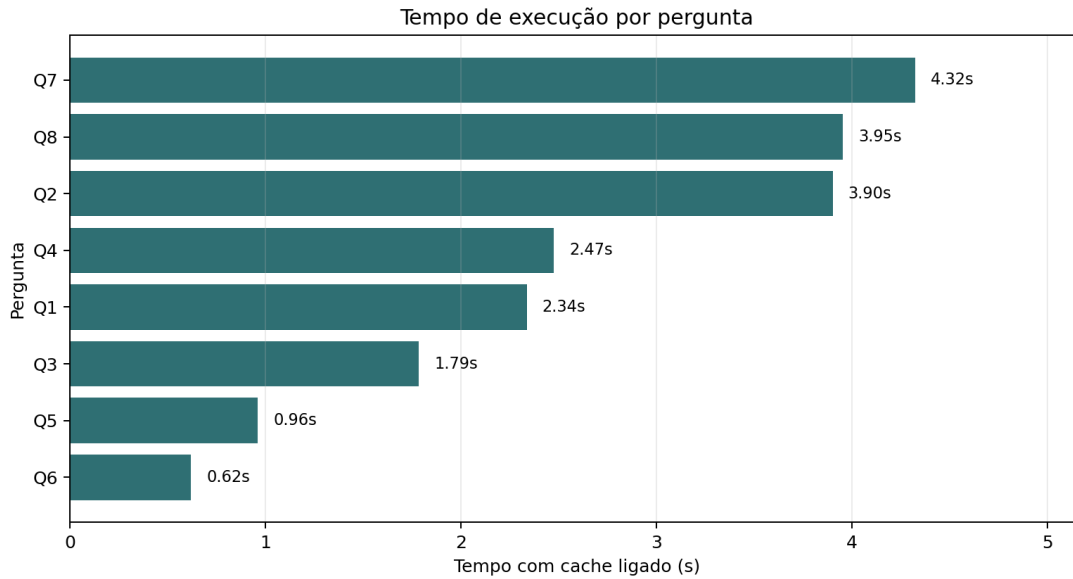


Figura 4: Tempo das perguntas com cache ligado. Os rótulos foram reduzidos para Q1 a Q8 para facilitar a leitura.

O maior ganho aparece nas perguntas que reutilizam bases limpas e joins. A Q6, por exemplo, depende do cruzamento entre temperatura e CO2. Com cache, o Spark reaproveita o resultado intermediário; sem cache, precisa refazer mais trabalho.

6.3 Benchmark local versus cluster LAN

A configuração principal do trabalho usa dois workers. No Compose local, os dois workers ficam no PC 1. No cluster LAN, um worker fica no PC 1 e outro no PC 2. O script `scripts/benchmark_cluster_modes.sh` mede tempo total, tempo de processamento Spark e overhead aproximado.

Tabela 10: Comparação entre execução local e cluster LAN

Modo	Workers	Wall (s)	Spark (s)	Overhead (s)
local-compose	2 workers no PC 1	158	35.4613	122.5387
lan-cluster	1 worker no PC 1 e 1 no PC 2	188	43.8593	144.1407

O modo local foi mais rápido nesta carga. O modo LAN distribuiu o processamento entre dois computadores, mas teve overhead maior.

Com esses valores:

```

speedup_lan_vs_local = 158 / 188 = 0.84
eficiencia_lan = 0.84 / 2 = 0.42
diferenca_de_overhead = 144.1407 - 122.5387 = 21.6020s

```

A conclusão prática é que o processamento foi distribuído, mas esta execução em LAN não foi mais rápida. A Spark UI confirmou dois workers vivos e executores em hosts diferentes. Mesmo assim, o custo de rede, Docker, submit, shuffle e sincronização superou o ganho de usar o PC 2.

6.4 Workers, recursos e escalabilidade

Os workers locais usam:

```
--cores 2
--memory 2G
```

Para testar mais recursos por worker, pode-se mudar para:

```
--cores 4
--memory 4G
```

Isso só faz sentido se a máquina tiver CPU e memória disponíveis. Mais recursos não garantem melhora se o gargalo estiver em leitura de disco, shuffle, driver, escrita final ou etapas pouco paralelizáveis.

No Compose atual, os workers estão nomeados como spark-worker-1 e spark-worker-2. Para três workers, duplica-se o bloco e cria-se spark-worker-3, com porta 8083:8081. Para quatro workers, cria-se spark-worker-4, com porta 8084:8081.

Se o Compose for refatorado para um serviço genérico spark-worker, a escala poderia ser feita com:

```
SPARK_WORKER_CORES=2 SPARK_WORKER_MEMORY=2G docker compose up -d --scale spark-
  worker=3
SPARK_WORKER_CORES=2 SPARK_WORKER_MEMORY=2G docker compose up -d --scale spark-
  worker=4
```

Para comparar N workers contra a base de dois workers:

```
speedup = tempo_2_workers / tempo_N_workers
eficiencia = speedup / (N / 2)
constante_de_rede_aproximada = network_orchestration_overhead
```

Os testes com três e quatro workers são uma extensão natural do experimento, mas não fazem parte da medição principal registrada aqui.

6.5 Por que mais workers podem não reduzir o tempo

Se três ou quatro workers não diminuam o tempo, os motivos mais prováveis são:

- dataset pequeno para o custo de distribuir;
- poucas partições;
- workers no mesmo PC competindo pelo mesmo disco;

- shuffle alto;
- escrita final serializada por `coalesce(1)`;
- driver virando gargalo;
- etapas com window functions sem `partitionBy`;
- custo de rede maior que o ganho de paralelismo.

Durante a execução apareceu o aviso:

```
WindowExec: No Partition Defined for Window operation
```

Isso indica que algumas janelas foram executadas sem particionamento explícito, fazendo o Spark mover dados para uma única partição. Esse trecho reduz o paralelismo e ajuda a explicar por que o ganho não é linear.

6.6 Conclusão a partir das medições

O projeto demonstra um pipeline Spark completo e distribuído: leitura de CSVs reais, limpeza, normalização, agregações, join entre temperatura e CO2, window functions, cache, MLlib, escrita de resultados e geração de gráficos.

A execução em dois PCs foi validada pela Spark UI com dois workers vivos e executores em hosts diferentes. A medição com base raw mostrou que distribuir não garante acelerar sempre. O Compose local terminou em 158 segundos, enquanto o cluster LAN terminou em 188 segundos. O modo LAN teve 21.6020 segundos a mais de overhead estimado.

A parte mais favorável do pipeline foi o cache: nas perguntas Q1 a Q8, o tempo somado caiu de 118.4348 segundos sem cache para 20.3530 segundos com cache, um ganho aproximado de 5.82x. Isso mostra que, para este trabalho, reaproveitar etapas intermediárias foi mais impactante do que apenas adicionar outro computador.

Portanto, a defesa técnica é: o cluster funcionou, os dados e o processamento foram distribuídos, mas o ganho de desempenho depende do tamanho da carga, do número de partições, do custo de shuffle, da rede e das partes serializadas do pipeline.